

LoveMatch

Development Phases & Architecture

A comprehensive, phase-by-phase record of every technical implementation, feature, database migration, security hardening measure and deployment pipeline built for the LoveMatch platform, followed by the proposed plan for the next phase of advanced backend work.

PLATFORM	PHASES DELIVERED	PHASES PROPOSED	STACK
LoveMatch	5	1	Django 5.x / Channels

Overview of Development Phases

Phase	Title	Primary Branch	Key Focus Areas
Phase 1	Enterprise Foundation & Security	feature/phase-1-foundation-security-db	Config decoupling, database models overhaul, security headers, data seeder
Phase 2	Matching Engine & Interactive Swiping	feature/phase-2-matching-swipe-engine	Gesture card swipe deck, 5-axis compatibility radar, date proposals
Phase 3	Real-Time WebSockets & AI Spark	feature/phase-3-realtime-chat-ai	Django Channels chat, typing indicators, Gemini AI icebreaker generator
Phase 4	Trust, Safety & Profile Gamification	feature/phase-4-safety-multiphoto-profile	Block/report system, 6-photo gallery, profile prompts, strength meter
Phase 5	Testing, Docker & CI/CD Pipeline	feature/phase-5-docker-ci-tests	Pytest automated test suite, Docker & Compose stack, GitHub Actions CI/CD
Phase 6	Advanced Backend Features	not yet branched – proposed	PostGIS geolocation, social sign-on, web push, premium tier, WebRTC calling

PHASE

01

Enterprise Foundation, Security & Database Overhaul

Branch `feature/phase-1-foundation-security-db` | Commit `76762e1`

Objectives

- Establish an enterprise-ready codebase with decoupled configurations.
- Overhaul data models to support full-scale matchmaking, astrological synergy, and trust & safety.
- Provide initial database seeding for interests and psychological compatibility questions.

Key Implementations

Environment & Security Decoupling `lovematch/settings.py`

- Integrated `python-decouple` and `django-database-url` to externalise all environment-specific configuration.
- Created `.env.example` carrying standard defaults and every configuration variable the project expects.
- Configured production security headers: `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS` and `X_FRAME_OPTIONS = 'DENY'`.
- Integrated `whitenoise` for static file compression and long-lived caching.

Core Database Schema Expansion `accounts/models.py`

- **Profile** — expanded with `bio`, `zodiac_sign`, `city`, `location_lat`, `location_lng`, `ghost_mode` (incognito browsing), `is_verified` and `profile_strength`.
- **InterestTag** — tag taxonomy categorised by Lifestyle, Sports, Creativity, Tech, Romance and more.
- **ProfilePhoto** — gallery photo model supporting multiple uploads with explicit ordering and primary avatar flags.
- **ProfilePrompt** — Hinge-style prompt questions paired with custom user responses.
- **BlockedUser** and **UserReport** — moderation models tracking block lists and flagged profile reports with categorised violation types.

Data Seeding Engine `accounts/management/commands/seed_data.py`

- Automated CLI command `python manage.py seed_data` seeds default interest tags and the multi-dimensional questionnaire datasets.

Django Admin Customisation `accounts/admin.py`

- Configured rich list displays, filters, search fields and inline editors for user profiles, photos, reports and prompts.

PHASE

02

Interactive Swipe Deck, Compatibility Radar & Date Proposals

Branch `feature/phase-2-matching-swipe-engine` | Commit `45f3de9`

Objectives

- Provide an engaging, modern swipe interface inspired by leading dating applications.
- Implement deep multi-dimensional compatibility algorithms, visualised through interactive radar charts.
- Build in-app date proposal functionality.

Key Implementations

Interactive Swipe Deck UI `templates/matching/discover_swipe.html`

- Touch and pointer gesture engine with smooth spring physics and rotation transforms.
- Action controls: Swipe Left (Pass), Swipe Right (Like), Swipe Up (Superlike) and Rewind.
- Celebration modal with dynamic confetti via `canvas-confetti`, triggered on mutual matches.

Multi-Dimensional Compatibility Engine `matching/services.py`

- Computes an overall score alongside a five-axis psychological and lifestyle breakdown:
 - Romance & Affection
 - Fun & Humour
 - Shared Values & Beliefs
 - Lifestyle Pace & Habits
 - Communication & Social Energy
- Integrates tag overlap multipliers and astrological zodiac elemental synergy (Fire, Earth, Air, Water).
- Rendered using `Chart.js` inside an interactive modal.

Date Proposals Subsystem `matching/models.py`, `matching/views.py`

- `DateProposal` model supporting curated date types: Dinner, Coffee, Adventure, Movie and Picnic.
- Life-cycle status handling across `PENDING`, `ACCEPTED`, `DECLINED` and `CANCELLED`.
- Notification and dashboard hooks for real-time tracking.

Discovery Filters

- Filter the candidate deck by age bracket, maximum distance, minimum compatibility percentage and shared tags.

PHASE

03

Real-Time WebSocket Chat, Live Indicators & AI Spark

Branch `feature/phase-3-realtime-chat-ai` | Commit `e845239` · `f9c54a0`

Objectives

- Deliver full-duplex, low-latency live messaging.
- Build live typing indicators, delivery checkmarks and read receipts.
- Embed Google Gemini AI to generate contextual conversation starters.

Key Implementations

Asynchronous WebSockets Architecture `lovematch/asgi.py, chat/consumers.py, chat/routing.py`

- ASGI application powered by `channels` and `Daphne` with a Redis channel-layer backend.
- `ChatConsumer` handles message broadcasts, real-time presence and typing status updates.

Advanced Chat Experience `templates/chat/conversation.html`

- WhatsApp / iMessage-style dual message bubbles with timestamps.
- Double checkmark read receipts — a single check marks Sent, a double check marks Read.
- Live animated “typing...” bubble indicator.
- Inline rendering of accepted and pending Date Proposal cards.

AI Spark Icebreaker Generator `chat/ai_service.py`

- Integrated Google Gemini AI through the official `google-genai` SDK.
- Compiles mutual hobbies, zodiac signs and bio keywords into tailored prompts that generate three witty, charming conversation starters.
- Includes a robust fallback heuristic generator for offline operation or when no API key is configured.

Message Reaction & Reply Model `chat/models.py`

- Message schema extended with `is_read`, `read_at`, `reply_to` and `MessageReaction` relations.

PHASE

04

Trust & Safety System, Multi-Photo Gallery & Profile Gamification

Branch `feature/phase-4-safety-multiphoto-profile` | Commit `b038cbe`

Objectives

- Create a safe, moderated community environment with intuitive blocking and reporting tools.
- Provide rich profile customisation with multiple photos and Hinge-style prompt cards.
- Gamify profile completeness with a dynamic strength meter.

Key Implementations

Trust & Safety Shield `accounts/views.py`, `templates/accounts/blocked_list.html`

- **User blocking** — instantly terminates active matches, removes the user from chat lists and blocks all future discovery.
- **Report ticket submission** — structured modal to report profiles for inappropriate behaviour, fake accounts, harassment or spam.
- **Blocked users manager** — dedicated view allowing users to review and manage their block list.
- Discovery queryset filtering guarantees blocked and ghost-mode users remain invisible.

Multi-Photo Gallery System `accounts/views.py`, `templates/accounts/profile_edit.html`

- Upload up to six profile photos with drag-and-drop support.
- Instant primary avatar switcher and individual photo deletion.

Profile Prompts & Answer Cards

- Pre-curated prompts such as “My ideal Sunday looks like...”, “A boundary I maintain...” and “Two truths and a lie...”.
- Displayed as styled cards on public profile views.

Gamified Profile Strength Meter

- Dynamic JavaScript and backend completeness scoring from 0% to 100%.
- Provides contextual recommendations to improve the score, for example “+15% Add at least 3 photos” and “+10% Answer a prompt”.

PHASE

05

Automated Testing Suite, Docker Containerisation & CI/CD

Branch `feature/phase-5-docker-ci-tests` | Commit `1ed5991`

Objectives

- Build an automated test suite across all critical business logic.
- Deliver multi-stage Docker containerisation and Docker Compose orchestration.
- Set up a continuous integration pipeline via GitHub Actions.

Key Implementations

Automated Test Suite `tests/`, `pytest.ini`

- `test_auth_and_profiles.py` — user registration, profile strength calculation, prompt updates and photo uploads.
- `test_matching_and_swiping.py` — swiping gestures, mutual match creation, compatibility radar calculations and discovery filters.
- `test_chat_and_ai.py` — real-time chat model operations, AI icebreaker generation and in-chat date planning.
- `test_safety.py` — block and report enforcement, message isolation and discovery exclusion verification.

Docker Containerisation `Dockerfile`, `.dockerignore`

- Multi-stage build based on `python:3.12-slim`.
- Configured a non-root execution user for runtime security.
- Automated `collectstatic` compilation during the build.
- Daphne ASGI web server entrypoint.

Multi-Container Orchestration `docker-compose.yml`

- `web` — Daphne ASGI server running the Django LoveMatch application.
- `db` — PostgreSQL 16 Alpine database with health checks and a persistent volume.
- `redis` — Redis 7 Alpine cache and message broker for WebSockets.

Continuous Integration Pipeline `.github/workflows/ci.yml`

- Automated workflow triggered on pushes and pull requests to `main`.
- Runs `flake8` lint checks, database migrations, and `pytest` with code coverage reporting.

PHASE

06

Advanced Backend Features

Implementation plan — proposed, awaiting review and sign-off

This section sets out the architecture and steps required to implement the five advanced features proposed for the LoveMatch platform before frontend production development begins. Nothing here has been built; the decisions below are blocking.

Decisions Required Before Implementation

WARNING**Database engine change (PostGIS)**

Moving to advanced geofencing requires switching the database engine from standard PostgreSQL to PostGIS. This means adding system-level dependencies (GDAL, GEOS) to the Docker container and altering the `location_lat` and `location_lng` fields to a `PointField`. This is a significant migration.

IMPORTANT**Third-party keys and integrations**

Social auth — `django-allauth` will be installed, but Google and Apple client IDs and secrets must be supplied through the Django admin or the environment before the providers function in production.

Web push — `django-webpush` will be implemented for browser notifications and will generate VAPID keys automatically.

OPEN QUESTION**Monetisation and premium tier**

Should a payment provider such as Stripe be integrated for handling subscriptions, or should this phase build only the database models (for example a `Subscription` model) and the access control logic (for example locking read receipts behind a paywall)?

Working assumption for this plan: internal models and logic only, leaving external Stripe integration to a dedicated billing phase unless directed otherwise.

Proposed Changes

1 Advanced Geolocation (PostGIS)

Upgrading the matching engine to support precise spatial queries, such as finding users within five miles.

MODIFY

Dockerfile

- Install PostGIS dependencies: `apt-get install -y binutils libproj-dev gdal-bin postgis`.

MODIFY

docker-compose.yml

- Change the db image from `postgres:16-alpine` to `postgis/postgis:16-3.4-alpine`.

MODIFY

lovematch/settings.py

- Update the DATABASES engine to `django.contrib.gis.db.backends.postgis`.

MODIFY

accounts/models.py

- Import from `django.contrib.gis.db` import models as `gis_models`.
- Add `location = gis_models.PointField(geography=True, null=True, blank=True)` to the Profile model.

MODIFY

matching/services.py

- Update `get_discover_candidates` to filter by distance using `D(mi=search_radius)`.

2 Social Sign-On (SSO)

Implementing Google and Apple login through `django-allauth`.

MODIFY

lovematch/settings.py

- Add `allauth`, `allauth.account`, `allauth.socialaccount` and `allauth.socialaccount.providers.google` to `INSTALLED_APPS`.
- Configure `AUTHENTICATION_BACKENDS` and the associated middleware.

MODIFY

lovematch/urls.py

- Include `path('accounts/', include('allauth.urls'))`.

3 Push Notifications (Web Push)

Allowing the application to send notifications when users receive matches or messages while offline.

MODIFY

lovematch/settings.py

- Add `webpush` to `INSTALLED_APPS`.

MODIFY

lovematch/urls.py

- Include `path('webpush/', include('webpush.urls'))`.

MODIFY

chat/consumers.py, matching/views.py

- Integrate `send_user_notification` to trigger push alerts on new matches and unread messages.

4 Premium / Monetisation Tier

Adding subscription models to restrict certain features, such as undo swipe or seeing who liked you.

NEW

accounts/models.py

- `UserSubscription` model with fields `user`, `plan_type` (Free, Gold, Platinum) and `active_until`.
- Add helper method `is_premium()` to the Profile model.

MODIFY

matching/views.py, chat/consumers.py

- Add access control checks for premium features, for example unlimited swipes.

5 Video / Audio Calling (WebRTC)

Adding a signaling server to facilitate peer-to-peer video calls for virtual dates.

MODIFY	chat/routing.py Add a WebSocket route for <code>/ws/call/<room_name>/</code>.
NEW	chat/call_consumers.py Create <code>CallConsumer</code> to handle WebRTC signaling events: <code>offer</code>, <code>answer</code>, <code>ice_candidate</code> and <code>end_call</code>.

6 Documentation

Keeping the phase record current alongside the implementation.

MODIFY	DEVELOPMENT_PHASES.md Add Phase 6: Advanced Backend Features (SSO, WebRTC, PostGIS, Push, Premium Tier).
--------	---

Verification Plan

Automated Tests

- Run `pytest` across the existing suites to confirm no regressions.
- Add new tests in `tests/test_advanced_features.py` covering subscription models, WebRTC signaling events and PostGIS distance querying logic.

Manual Verification

- Rebuild the Docker stack with `docker compose build` to test the new PostGIS image.
- Test the WebRTC connection payload passing via a mock WebSocket client.
- Test `is_premium` restrictions on the swipe endpoints.

Tech Stack & Architecture

Backend Framework	Django 5.x with Django Channels (ASGI)
ASGI Server	Daphne
Database	PostgreSQL (production) / SQLite3 (development)
Cache & Channel Layer	Redis 7
AI Model	Google Gemini AI via the official google-genai SDK
Frontend	Bootstrap 5.3, Vanilla ES6 JavaScript, Chart.js, Canvas-Confetti, CSS3 animations
Testing	Pytest, Pytest-Django, Pytest-Cov
DevOps	Docker, Docker Compose, WhiteNoise, GitHub Actions CI/CD

Branch & Commit Reference

Phase	Primary Branch	Commit
Phase 1	feature/phase-1-foundation-security-db	76762e1
Phase 2	feature/phase-2-matching-swipe-engine	45f3de9
Phase 3	feature/phase-3-realtime-chat-ai	e845239 · f9c54a0
Phase 4	feature/phase-4-safety-multiphoto-profile	b038cbe
Phase 5	feature/phase-5-docker-ci-tests	1ed5991
Phase 6	not yet branched	proposed

End of document

Five phases delivered across foundation and security, the matching engine, real-time messaging, trust and safety, and the automated testing and deployment pipeline. Phase 6 is proposed and awaiting sign-off on the PostGIS migration, third-party credentials and the monetisation scope.